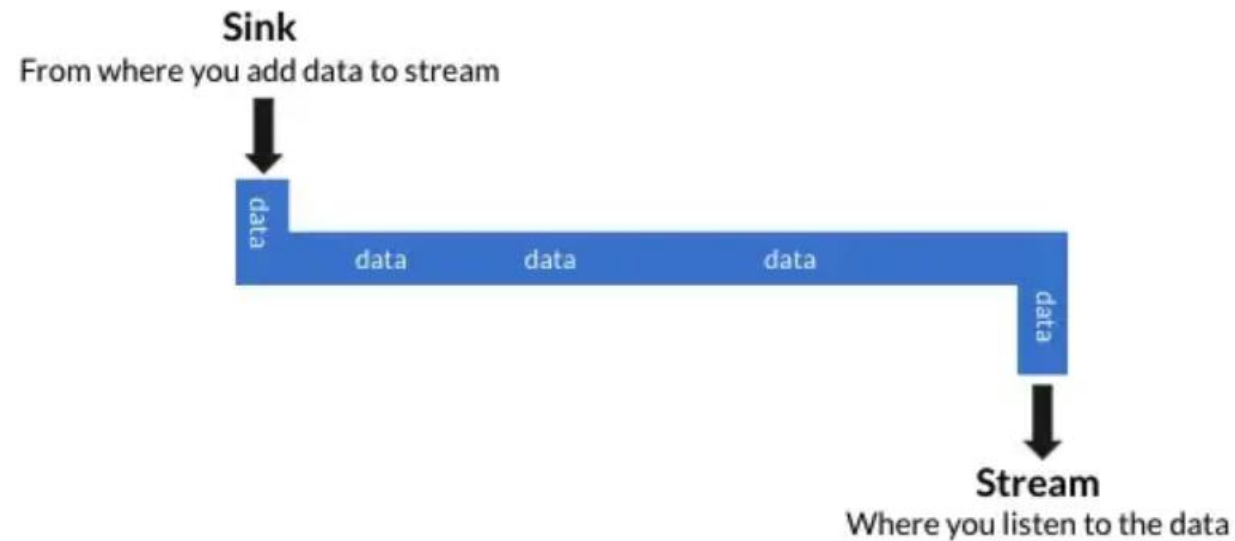


MOBILE DEVELOPMENT 2

W10 – Streams, Firebase SDK





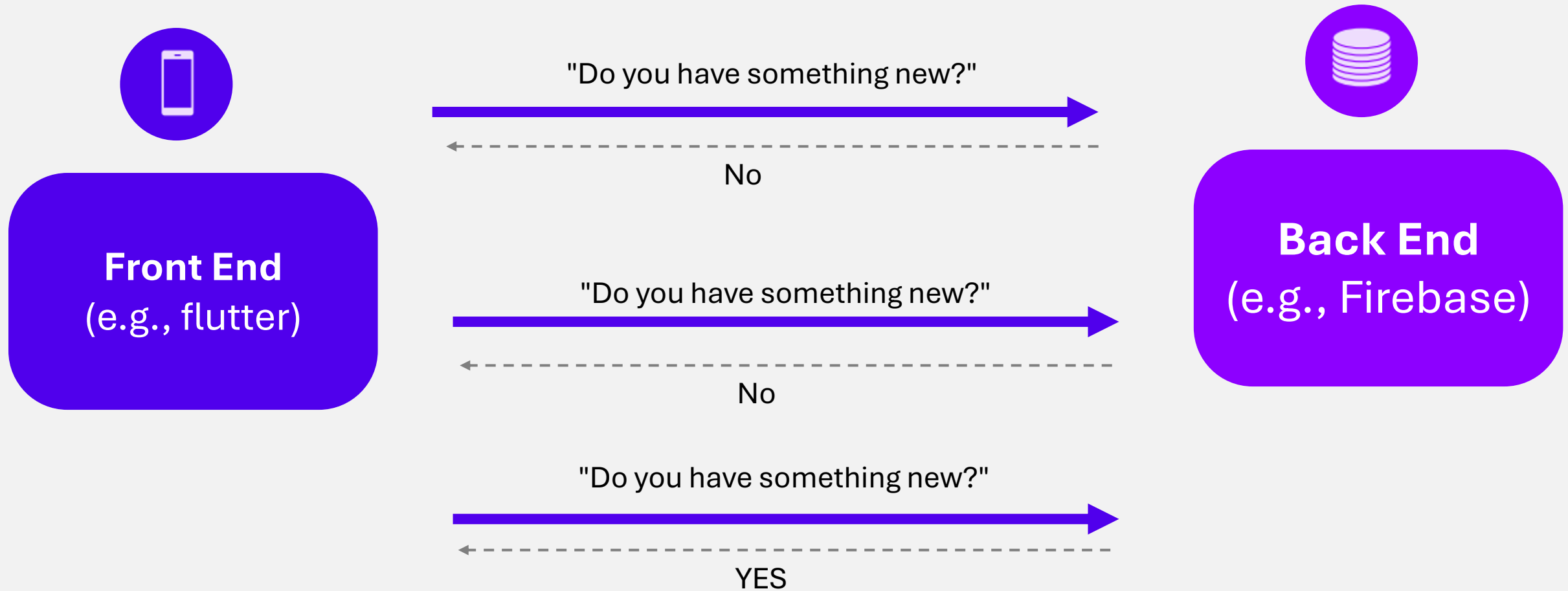
Course Objectives



- ✓ Understand the link between **Future** and **Stream**
- ✓ Build a **download progress stream**
- ✓ Consume streams in Flutter (**StreamBuilder**)
- ✓ Use streams with **real-time database updates** (Firestore / Storage)

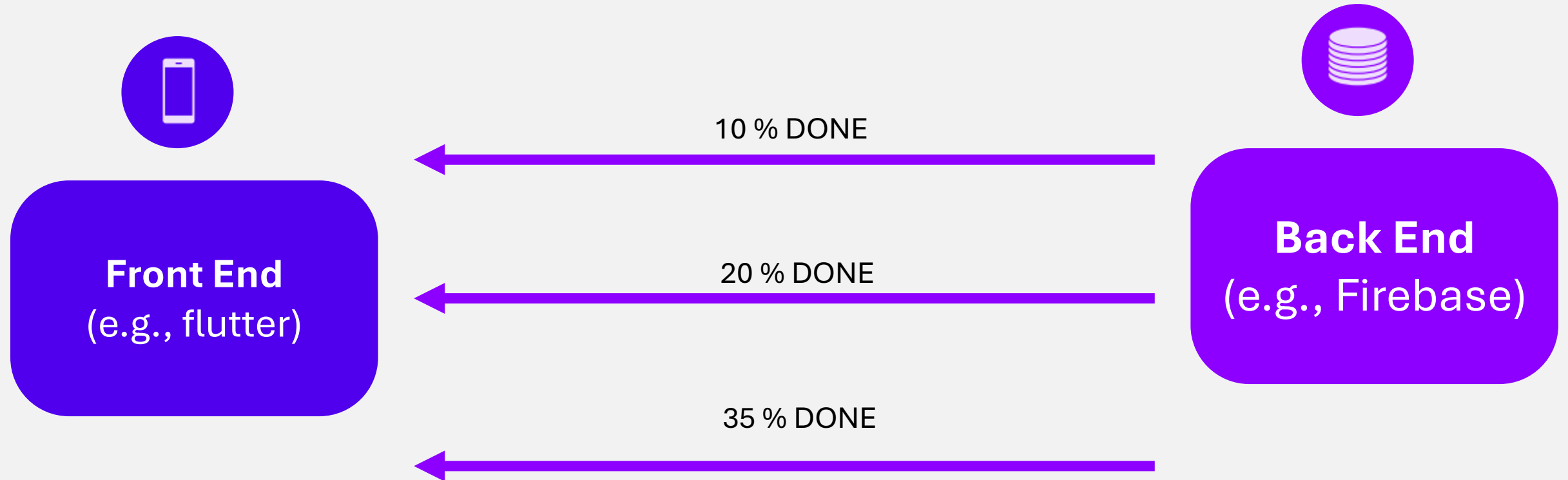
Pull Model

In a **pull model**, Client is in control and we need repeated requests (polling)



Push Model

In a **pull model**, Server is in control and the server send data to client when available

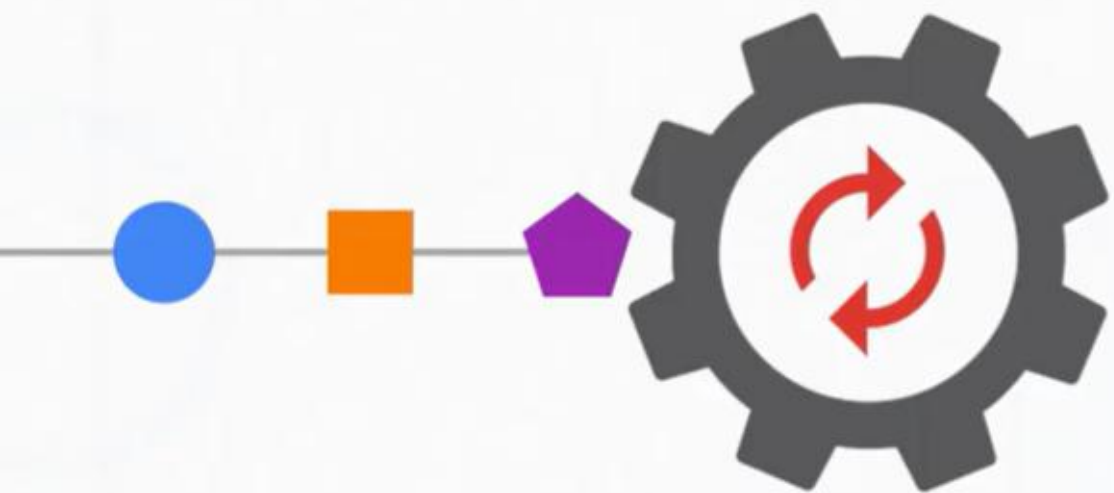




Think of examples where
the **app does not control** when data updates !!

- ✓ Write on paper a few ideas
- ✓ Share in groups (4)
- ✓ Share in telegram





What is a **Stream** ?

A **Stream** is a **sequence of asynchronous values** over time.

SYNCHRONOUS	Int	Iterator<int>
ASYNCHRONOUS	Future<int>	Stream <int>

REST API calls

Progress, real time updates..

Create a stream

We create a stream controller that send (add) value (ex : int) into a stream

```
final controller = StreamController<int>();

int counter = 0;

Timer.periodic(Duration(seconds: 1), (timer) {
  counter++;
  controller.add(counter); ← Send the value into the stream

  if (counter == 5) {
    timer.cancel();
    controller.close(); ← Close the stream
  }
});
```



Listen to streams

We create a stream controller that send (add) value (ex : int) into a stream

Adds a subscription to this stream

A new event is triggered

The stream is closed

```
controller.stream.listen(  
    (newValue) => print("Received $newValue"),  
    onDone: () => print("done"),  
);
```



Upgrade the Download Stream

- 1 - Create a dedicated **DownloadProgress** to carry the progress + the timestamp (datetime)
- 2 - Change the stream to send DownloadProgress objects instead of just integers.
- 3 - Update startDownload() to send the **current time** along with **progress** (DateTime.now)
- 4 - **Listen** to the stream
- 5 - For each new value, compute the **estimated remaining time**
💡 *Hint: Keep the first timestamp to calculate time difference.*
- 6 Print something like:
Progress: 30% | Remaining: 700ms
- 7 – finally **simulate an error** every 10 iterations

Start Code

```
class DownloadService {  
    final StreamController<int> _controller = StreamController();  
  
    Stream<int> get stream => _controller.stream;  
  
    void startDownload() async {  
        for (int i = 0; i <= 100; i += 10) {  
            await Future.delayed(Duration(milliseconds: 300));  
            _controller.add(i);  
        }  
        _controller.close();  
    }  
}
```

Manipulate Streams

Streams are **reactive** — we can **transform** and **filter** data as it arrives

```
NumberCreator().stream  
    .where((i) => i % 2 == 0)  
    .map((i) => 'String $i')  
    .listen(print);
```



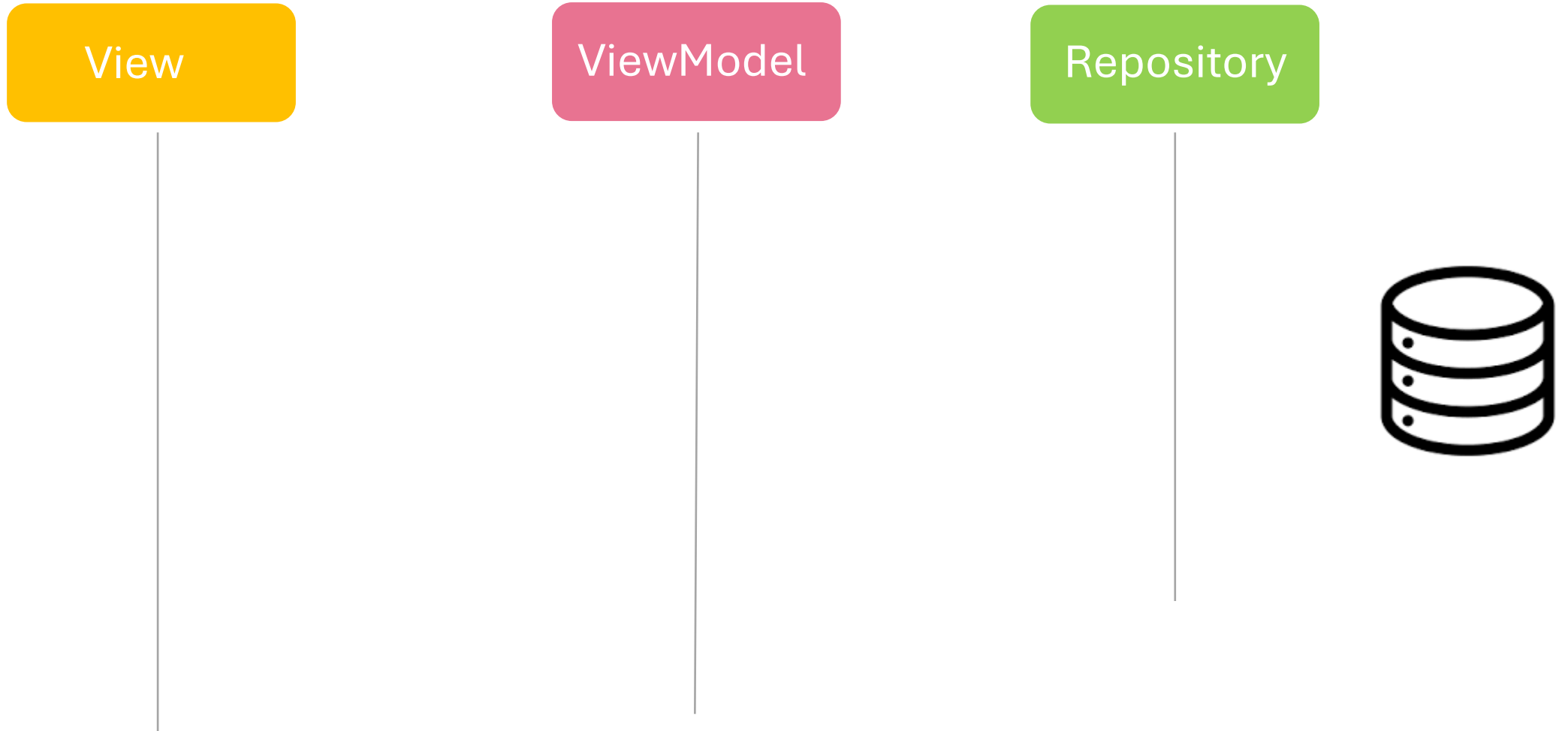
We receive a Stream
of STRING every
even numbers

Use a **StreamBuilder** to handle Stream

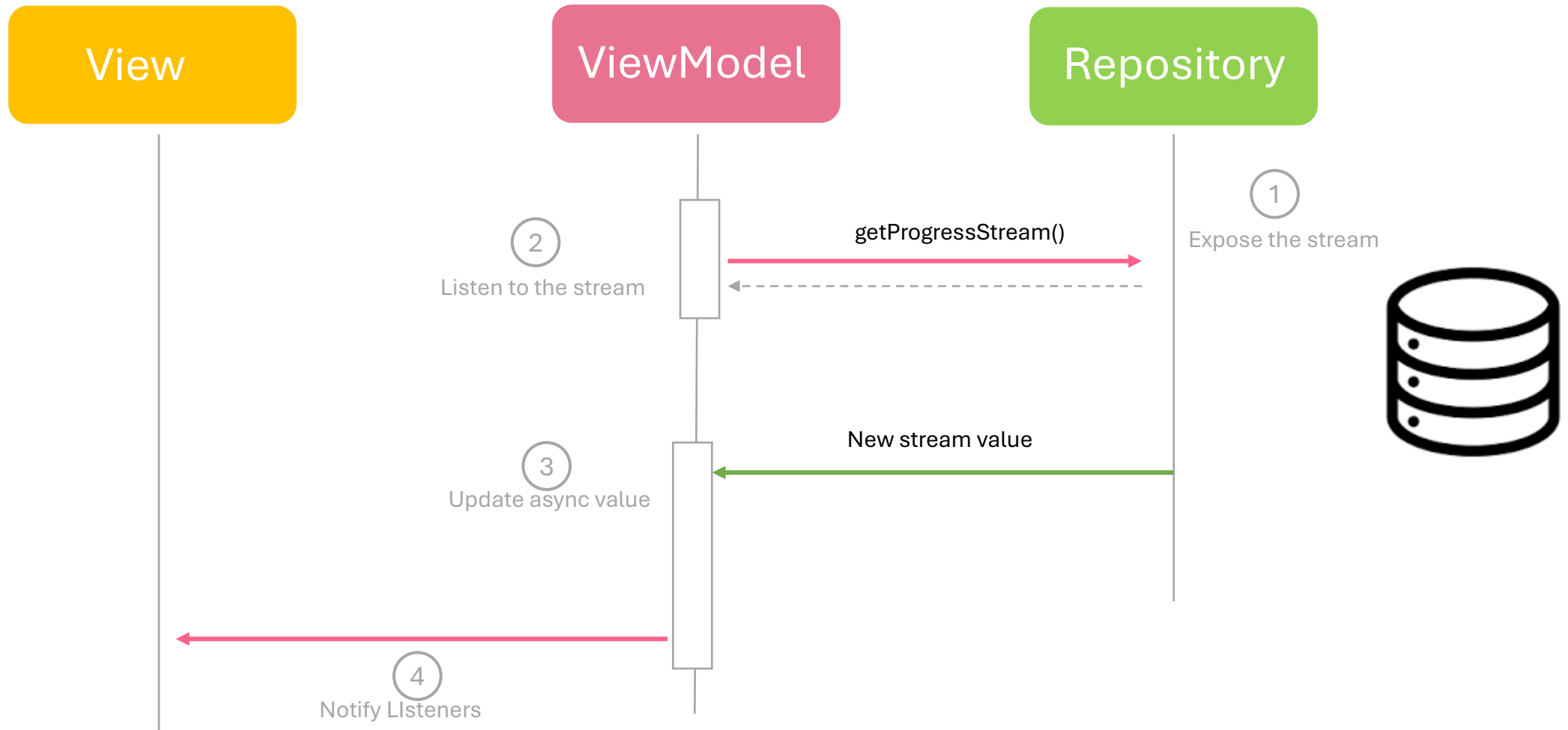
StreamBuilder rebuilds UI every time the stream emits a value.

```
StreamBuilder<int>(  
  stream: downloadService.downloadFile(),  
  builder: (context, snapshot) {  
    if (!snapshot.hasData) {  
      return CircularProgressIndicator();  
    }  
  
    return Column(  
      children: [  
        LinearProgressIndicator(value: snapshot.data! / 100),  
        Text("${snapshot.data}%"),  
      ],  
    );  
  },  
);
```

How to deal with **Stream** in our Flutter architecture?



How to deal with **Stream** in our Flutter architecture?



How to deal with **Stream** in our Flutter architecture?

ViewModel

```
class DownloadViewModel extends ChangeNotifier {  
  final DownloadRepository repository;  
  int progress = 0;  
  
  DownloadViewModel(this.repository) {  
    repository.progressStream.listen((value) {  
      progress = value;  
      notifyListeners(); // Update UI  
    });  
  }  
  
  void startDownload() => repository.startDownload();  
}
```

View

```
Widget build(BuildContext context) {  
  
  return Column(  
    children: [  
      Text("Progress: ${vm.progress}%"),  
      ElevatedButton(  
        onPressed: vm.startDownload,  
        child: Text("Start Download"),  
      ),  
    ],  
  );  
}
```

Upgrade the **Download App** (Week 4)

- Refactor the previous downloader app to use Repository → ViewModel → UI architecture
- Make the **ViewModel** listen to the **Stream** from the repository
- Use **Provider** / **ChangeNotifier** to notify the UI
- **BONUS** : Add a “cancel download” button while the resource is downloading





Streams with **Firestore**

Firestore is a cloud **NoSQL database** that stores data in collections and documents and syncs it in real time.

```
Stream<List<Task>> getTasks() {  
  return FirebaseFirestore.instance  
    .collection('tasks')  
    .snapshots() ← Stream<QuerySnapshot>  
    .map((snapshot) {  
      return snapshot.docs.map((doc) {  
        return Task.fromJson(doc.data()); ← Stream<List<Task>>  
      }).toList();  
    });  
}
```

The Firestore client exposes data as Streams

Streams with FireBase Storage



Firebase Storage is a cloud service for **storing** files like images, videos...

```
final fileRef = FirebaseStorage.instance.ref('uploads/myfile.jpg');  
final uploadTask = fileRef.putFile(file);
```

```
uploadTask.snapshotEvents.listen((snap) {  
    final progress = snap.bytesTransferred / snap.totalBytes * 100;  
    print('Upload: ${progress.toStringAsFixed(1)}%');  
});
```

```
uploadTask.whenComplete(() async {  
    print('Done! URL: ${await fileRef.getDownloadURL()}');  
});
```

snapshotEvents is a **Stream<TaskSnapshot>**
letting you react to progress in real time.